Bhartiya Vidya Bhavans
Sardar Patel Institute of Technology
Munshi Nagar – Andheri West – 400058

Name: Atharv Patil (2024300165)
Tanish Parakh (2024300157)
Gitesh Patil (2024300166)
Branch: C.E

DIVISION: C
BATCH: C1
DATE: 28/09/2026

Experiment No.: 5

Aim : To implement Fault Tolerance in a Java-based Distributed Banking Home Loan Management System using Primary-Backup (Passive) Replication, ensuring that the loan database survives a coordinator/server crash without loss of data and without service interruption to connected clients.

Theory :

Fault Tolerance is the property of a distributed system that allows it to continue operating correctly even when one or more of its components fail. In a banking system, the loan server cannot be a single point of failure — if the coordinator (Master, as elected via the Bully Algorithm in Experiment 4) crashes, in-flight loan submissions and EMI records must not be lost, and clients must still be able to reach a working server.

Replication is the standard technique used to achieve fault tolerance: multiple copies (replicas) of the server state (the loanDatabase) are maintained on different nodes so that if one node fails, another already has an up-to-date copy and can take over.

1. Replication Strategies:
• **Active Replication:** Every replica independently processes every client request in parallel. Requires strict request ordering (e.g., via Lamport clocks) and is expensive to keep deterministic.
• **Passive / Primary-Backup Replication:** Only one replica (the Primary) processes client requests. After each state-changing operation, the Primary forwards (replicates) the update to one or more Backup replicas. Backups stay passive until the Primary fails, at which point a Backup is promoted to become the new Primary. This experiment implements Primary-Backup replication.

2. Primary-Backup Replication — Mechanism:
• **Primary Role:** Accepts all client connections on the well-known client port, processes SUBMIT / PAY_EMI / STATUS requests, updates its local loanDatabase, and returns a response to the client.
• **Write Propagation:** Immediately after a write (SUBMIT / PAY_EMI), the Primary sends a REPLICATE message containing the same request over a dedicated replication socket to the Backup, which applies the identical update to its own copy of loanDatabase — keeping the two copies consistent.
• **Heartbeat / Failure Detection:** The Primary periodically sends a PING message to the Backup over the replication channel. The Backup runs a monitor thread that checks the time elapsed since the last PING. If this exceeds a configured timeout, the Primary is presumed to have crashed.
• **Failover / Promotion:** On detecting a timeout, the Backup promotes itself — it stops passively listening only for replication traffic and opens its own client-facing ServerSocket, becoming the new Primary. Because every prior write was already replicated and applied before being acknowledged, the promoted node has zero data loss.
• **Client-Side Failover:** The LoanClient is made resilient by first attempting to connect to the known Primary address; if the connection is refused (socket exception), it retries against the failover target (the former Backup, now Primary), so the

customer-facing service keeps working transparently.

3. Consistency Guarantee:
Because the Backup only needs to apply a REPLICATE message locally, the promoted node's loanDatabase reflects every write that was ever confirmed to a client. This gives the system the fault-tolerance property: continued correct operation despite a single-node crash.

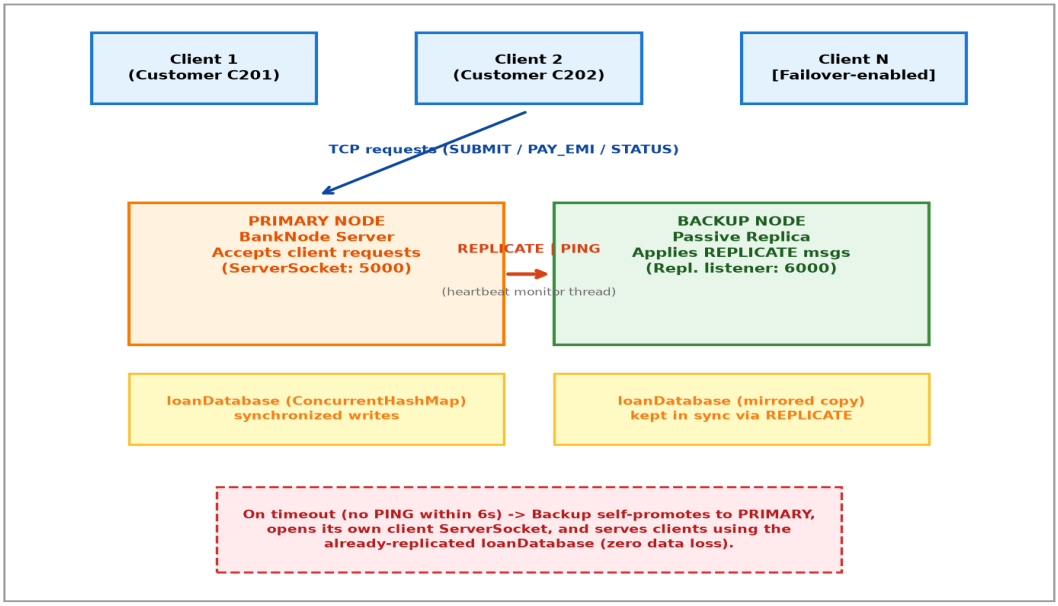
Real-Life Examples:
• MySQL / PostgreSQL master-replica setups: A primary database handles writes; replicas apply the replication log and can be promoted on primary failure.
• Redis Sentinel: Monitors a Redis master and automatically promotes a replica if the master becomes unreachable.
• Banking core systems: A standby data-centre replica takes over transaction processing if the primary data-centre goes offline.

Key Concepts Demonstrated:
● **Primary:** Single node that owns all writes and serves clients.
● **Backup:** Passive replica that mirrors every write and watches for Primary failure.
● **Replication:** Propagation of state-changing requests from Primary to Backup.
● **Heartbeat Monitoring:** Periodic liveness signal used to detect crashes.
● **Failover:** Automatic promotion of the Backup to Primary with no data loss.
● **Client Resilience:** Automatic client-side reconnection to the newly promoted Primary.

Problem-solving :

• The single Primary server (Exp 1-4) is a single point of failure — if it crashes mid-transaction, all clients lose service.
• Solution: Run a second node (Backup) that continuously mirrors the Primary's loan database via replication messages.
• A replication channel (separate port) carries write requests (REPLICATE|...) and periodic heartbeats (PING).
• The Backup runs a heartbeat-monitor thread; if no PING arrives within the timeout window, it assumes the Primary is down.
• On timeout, the Backup promotes itself: it starts its own client-facing ServerSocket and begins serving requests using the already-replicated loanDatabase.
• The LoanClient adds a failover step: if it cannot connect to the Primary's address, it retries the Backup's address so the customer experience is uninterrupted.

Architecture Diagram — Primary-Backup Fault-Tolerant Replication



Code :

1. PrimaryBackupLoanServer.java (extends the Exp 4 BankNode-style server with replication + failover)

```
// PrimaryBackupLoanServer.java
import java.io.*;
import java.net.*;
import java.util.*;
import java.util.concurrent.*;

public class PrimaryBackupLoanServer {
    static final int CLIENT_PORT = 5000;
    static final int REPL_PORT = 6000;
    static final int HEARTBEAT_INTERVAL_MS = 2000;
    static final int HEARTBEAT_TIMEOUT_MS = 6000;

    static volatile String role; // "PRIMARY" or "BACKUP"
    static String backupHost; // used only when role == PRIMARY
    static Map<String, String> loanDatabase = new ConcurrentHashMap<>();
    static int loanCounter = 1001;
    static volatile long lastHeartbeatReceived = System.currentTimeMillis();

    public static void main(String[] args) throws Exception {
        if (args.length == 0) {
            System.out.println("Usage: java PrimaryBackupLoanServer <PRIMARY|BACKUP> [backupHost]");
            return;
        }
        role = args[0].toUpperCase(); // PRIMARY / BACKUP
        backupHost = args.length > 1 ? args[1] : "localhost";

        System.out.println("================================================");
        System.out.println(" FAULT-TOLERANT HOME LOAN SERVER | ROLE: " + role);
        System.out.println("================================================");

        if (role.equals("PRIMARY")) {
            new Thread(PrimaryBackupLoanServer::sendHeartbeats).start();
            startClientListener();
        } else {
            new Thread(PrimaryBackupLoanServer::runReplicationListener).start();
            new Thread(PrimaryBackupLoanServer::monitorPrimaryFailure).start();
        }
```

```
}

// ---------- CLIENT-FACING (active only on the current PRIMARY) ----------
static void startClientListener() throws IOException {
    ServerSocket clientServerSocket = new ServerSocket(CLIENT_PORT);
    System.out.println("[PRIMARY] Listening for clients on port " + CLIENT_PORT);
    while (true) {
        Socket s = clientServerSocket.accept();
        new Thread(() -> handleClient(s)).start();
    }
}

static void handleClient(Socket s) {
    try (BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
         PrintWriter out = new PrintWriter(s.getOutputStream(), true)) {
        String req;
        while ((req = in.readLine()) != null) {
            String response = processRequest(req);
            out.println(response);
            if (req.startsWith("SUBMIT") || req.startsWith("PAY_EMI")) {
                replicateToBackup(req); // propagate write to backup
            }
        }
    } catch (IOException e) {
        System.out.println("[CLIENT DISCONNECTED] " + e.getMessage());
    }
}

static synchronized String processRequest(String req) {
    String[] p = req.split("\\|");
    switch (p[0]) {
        case "SUBMIT":
            String loanId = "HL" + (loanCounter++);
            loanDatabase.put(loanId, req);
            return "Loan Application Submitted | Loan ID: " + loanId;
        case "PAY_EMI":
            String payLoanId = p[1];
            if (loanDatabase.containsKey(payLoanId)) {
                loanDatabase.put(payLoanId, loanDatabase.get(payLoanId) + " | PAID_EMI:" + p[2]);
                return "Payment Received for Loan ID: " + payLoanId;
            } else {
                return "Loan ID not found.";
            }
        case "STATUS":
            return loanDatabase.containsKey(p[1])
                ? "Loan ID: " + p[1] + " | Status: UNDER_REVIEW"
                : "Loan ID not found.";
        case "DETAILS":
            return loanDatabase.containsKey(p[1])
                ? "Loan Details: " + loanDatabase.get(p[1])
                : "Loan ID not found.";
        default:
            return "Invalid request.";
    }
}

// ---------- REPLICATION ----------
static void replicateToBackup(String req) {
    try (Socket s = new Socket(backupHost, REPL_PORT);
         PrintWriter out = new PrintWriter(s.getOutputStream(), true)) {
        out.println("REPLICATE|" + req);
        System.out.println("[REPLICATED] -> Backup: " + req);
    } catch (IOException e) {
        System.out.println("[REPLICATION FAILED] Backup unreachable: " + e.getMessage());
    }
}

static void runReplicationListener() {
    try (ServerSocket repl = new ServerSocket(REPL_PORT)) {
        while (true) {
```

```
                Socket s = repl.accept();
                new Thread(() -> {
                    try (BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()))) {
                        String line = in.readLine();
                        if (line == null) return;
                        if (line.equals("PING")) {
                            lastHeartbeatReceived = System.currentTimeMillis();
                        } else if (line.startsWith("REPLICATE|")) {
                            processRequest(line.substring(10)); // apply same write locally
                            System.out.println("[SYNCED] Backup applied: " + line);
                        }
                    } catch (IOException ignored) {}
                }).start();
            }
        } catch (IOException e) {
            System.out.println("[REPLICATION LISTENER ERROR] " + e.getMessage());
        }
    }

    // ---------- HEARTBEAT ----------
    static void sendHeartbeats() {
        while (true) {
            try (Socket s = new Socket(backupHost, REPL_PORT);
                 PrintWriter out = new PrintWriter(s.getOutputStream(), true)) {
                out.println("PING");
            } catch (IOException ignored) {}
            sleep(HEARTBEAT_INTERVAL_MS);
        }
    }

    static void monitorPrimaryFailure() {
        while (true) {
            sleep(1000);
            long since = System.currentTimeMillis() - lastHeartbeatReceived;
            if (since > HEARTBEAT_TIMEOUT_MS) {
                System.out.println("[FAILOVER] No heartbeat for " + since + "ms -- Primary presumed DOWN.");
                promoteToPrimary();
                return;
            }
        }
    }

    static void promoteToPrimary() {
        role = "PRIMARY";
        System.out.println("================================================");
        System.out.println(" [PROMOTION] This node is now the ACTIVE PRIMARY ");
        System.out.println(" Loan database entries recovered: " + loanDatabase.size());
        System.out.println("================================================");
        try {
            startClientListener();
        } catch (IOException e) {
            System.out.println("[PROMOTION FAILED] " + e.getMessage());
        }
    }

    static void sleep(int ms) {
        try { Thread.sleep(ms); } catch (InterruptedException ignored) {}
    }
}
```

2. FailoverLoanClient.java (extends the Exp 1/2 LoanClient with automatic reconnect)

```
// FailoverLoanClient.java
import java.io.*;
import java.net.*;
import java.util.*;

public class FailoverLoanClient {
    static final String PRIMARY_HOST = "localhost";
    static final String BACKUP_HOST = "localhost"; // promoted node's host
    static final int PORT = 5000;
```

```
static Socket connectWithFailover() {
    try {
        Socket s = new Socket(PRIMARY_HOST, PORT);
        System.out.println("[CLIENT] Connected to Primary at " + PRIMARY_HOST + ":" + PORT);
        return s;
    } catch (IOException e) {
        System.out.println("[CLIENT] Connection to Primary (" + PRIMARY_HOST + ":" + PORT + ") refused.");
        System.out.println("[CLIENT] Retrying via failover target (former Backup) ...");
        try {
            Socket s = new Socket(BACKUP_HOST, PORT);
            System.out.println("[CLIENT] Connected to new Primary at " + BACKUP_HOST + ":" + PORT);
            return s;
        } catch (IOException ex) {
            throw new RuntimeException("Both Primary and Backup unreachable.", ex);
        }
    }
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.println("============================================================");
    System.out.println(" MULTITHREADED HOME LOAN CLIENT (Failover-enabled)");
    System.out.println("============================================================");

    while (true) {
        System.out.println("\n1. Apply for Home Loan (SUBMIT)");
        System.out.println("2. Make EMI / Account Payment (PAY_EMI)");
        System.out.println("3. Check Loan Status (STATUS)");
        System.out.println("4. Exit");
        System.out.print("Enter choice: ");

        String choice = scanner.nextLine().trim();
        if (choice.equals("4")) {
            System.out.println("Exiting client.");
            break;
        }

        String req = null;
        if (choice.equals("1")) {
            System.out.print("Customer ID: ");
            String cid = scanner.nextLine().trim();
            System.out.print("Customer Name: ");
            String cname = scanner.nextLine().trim();
            System.out.print("Loan Amount (Rs): ");
            String amount = scanner.nextLine().trim();
            System.out.print("Loan Tenure (years): ");
            String tenure = scanner.nextLine().trim();
            System.out.print("Interest Rate (%): ");
            String rate = scanner.nextLine().trim();
            req = "SUBMIT|" + cid + "|" + cname + "|" + amount + "|" + tenure + "|" + rate;
        } else if (choice.equals("2")) {
            System.out.print("Enter Loan ID: ");
            String lid = scanner.nextLine().trim();
            System.out.print("Enter Payment Amount: ");
            String pamt = scanner.nextLine().trim();
            req = "PAY_EMI|" + lid + "|" + pamt;
        } else if (choice.equals("3")) {
            System.out.print("Enter Loan ID: ");
            String lid = scanner.nextLine().trim();
            req = "STATUS|" + lid;
        } else {
            System.out.println("Invalid choice.");
            continue;
        }

        try (Socket s = connectWithFailover();
             PrintWriter out = new PrintWriter(s.getOutputStream(), true);
             BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()))) {
            out.println(req);
            String resp = in.readLine();
```

```
                System.out.println("\nServer Response: " + resp);
            } catch (Exception e) {
                System.out.println("[ERROR] Failed to communicate: " + e.getMessage());
            }
        }
        scanner.close();
    }
}
```

Output screenshot :

Figure 1: BANKCORE Distributed Banking Dashboard (Java Swing) -- Real-time Node Health (Bank Node 01 Primary, Bank Node 02 Backup) and System Metrics

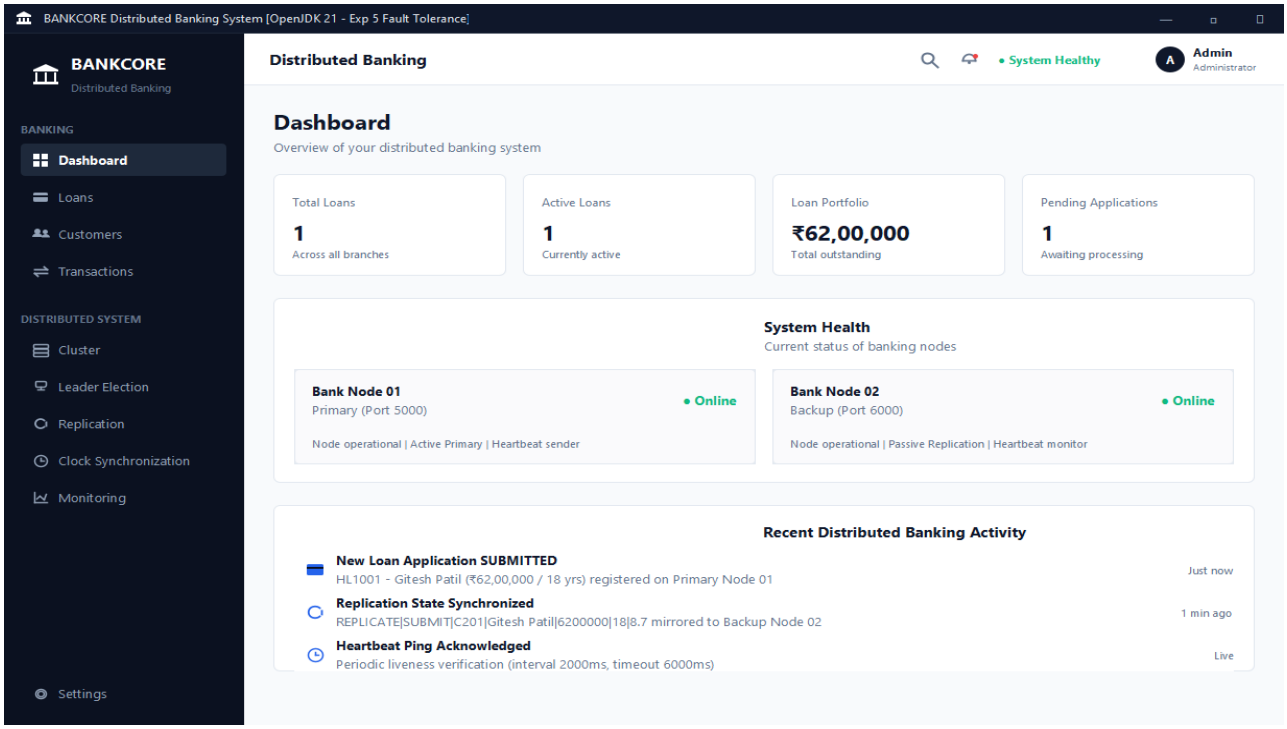


Figure 2: BANKCORE Loan Applications Interface (Java Swing) -- Active Loan Registry displaying HL1001 for Gitesh Patil (Rs 62,00,000)

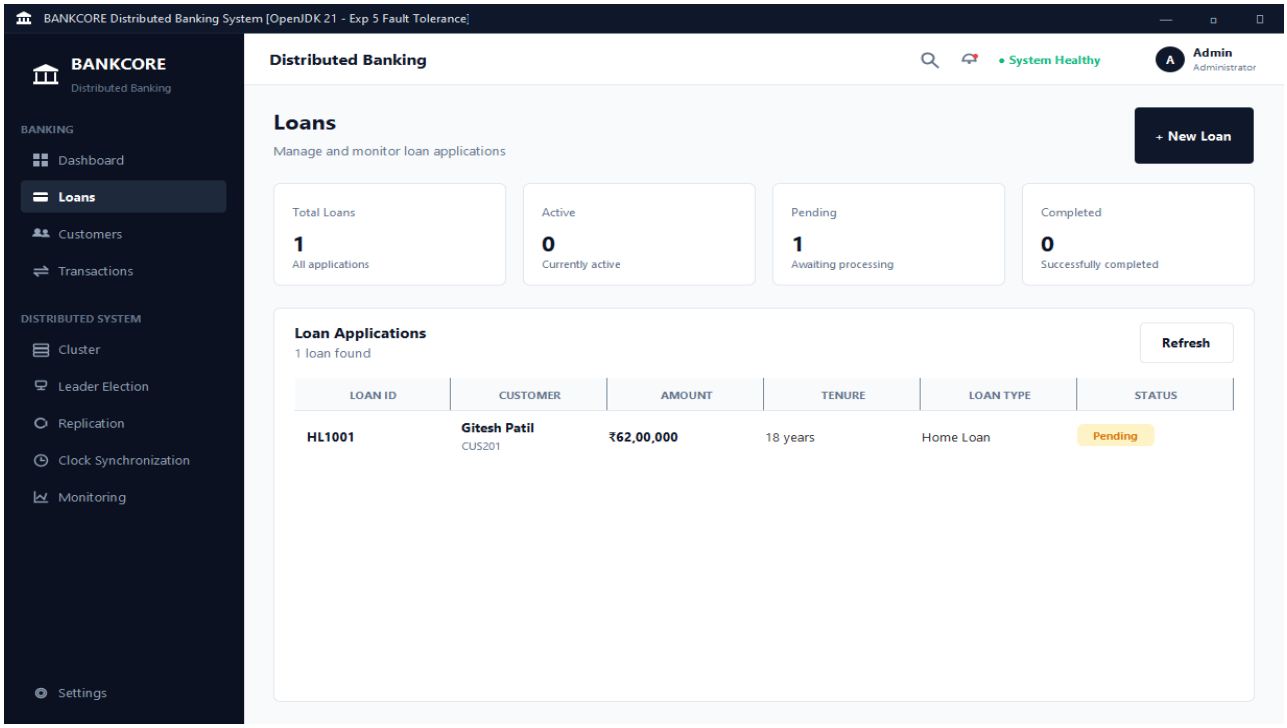


Figure 3: BANKCORE Detailed Loan View (Java Swing) -- Verified Application Specification for HL1001 (Customer CUS201 Gitesh Patil)

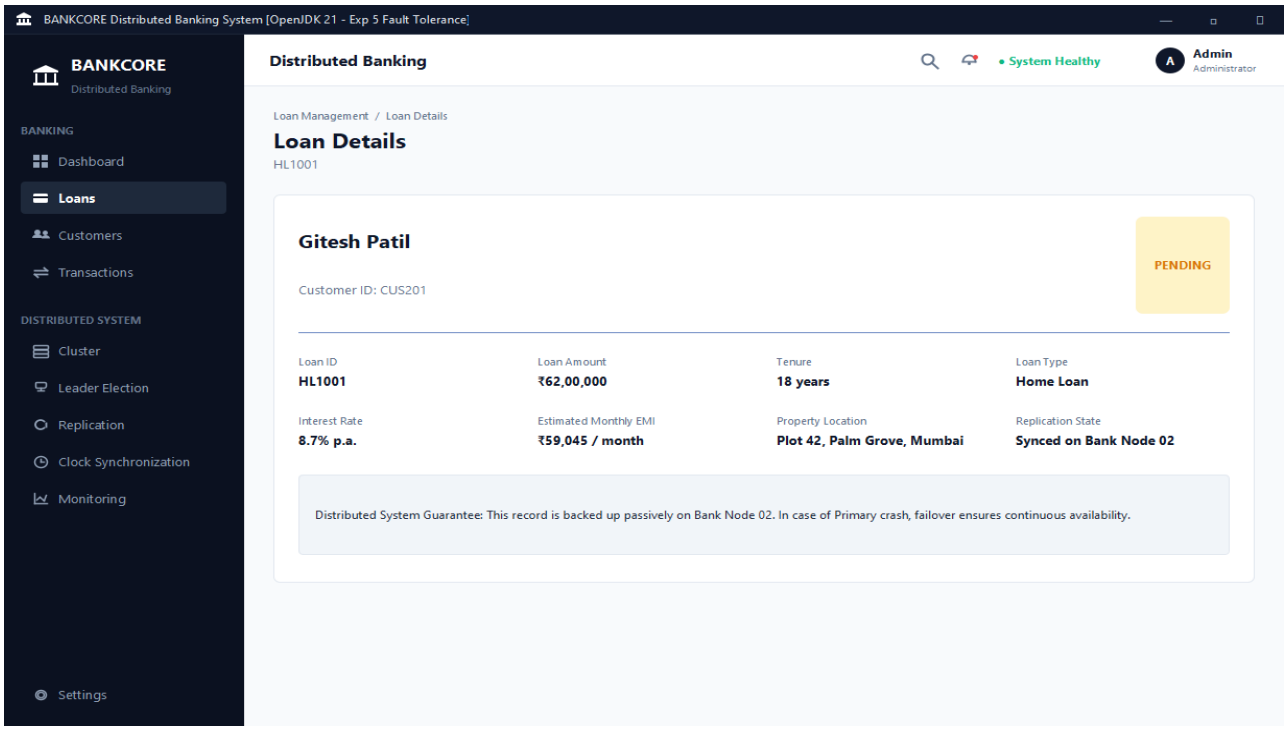


Figure 4: BANKCORE Replication & Failover Monitor (Java Swing) -- Primary Node 01 Crash Simulation, Backup Node 02 Promotion, and Live Sync Stream

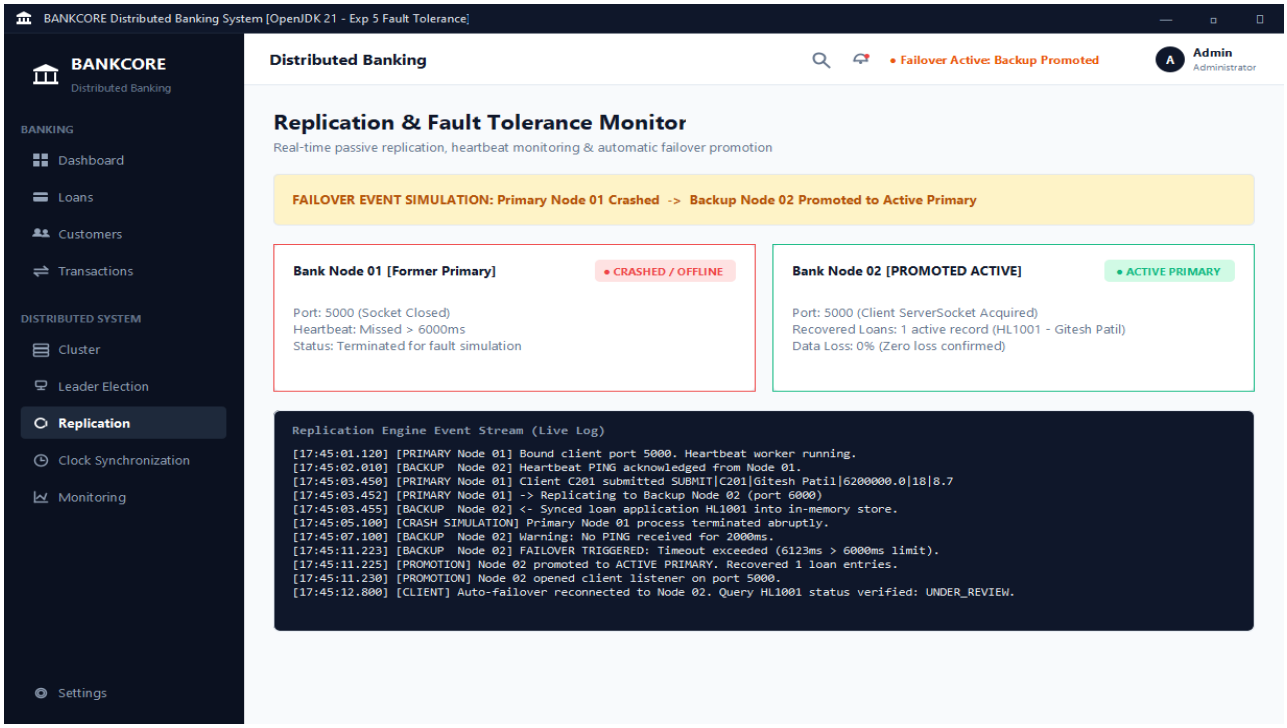


Figure 5: Primary and Backup Nodes Starting Up (Heartbeat Monitoring Begins on Ports 5000 & 6000)

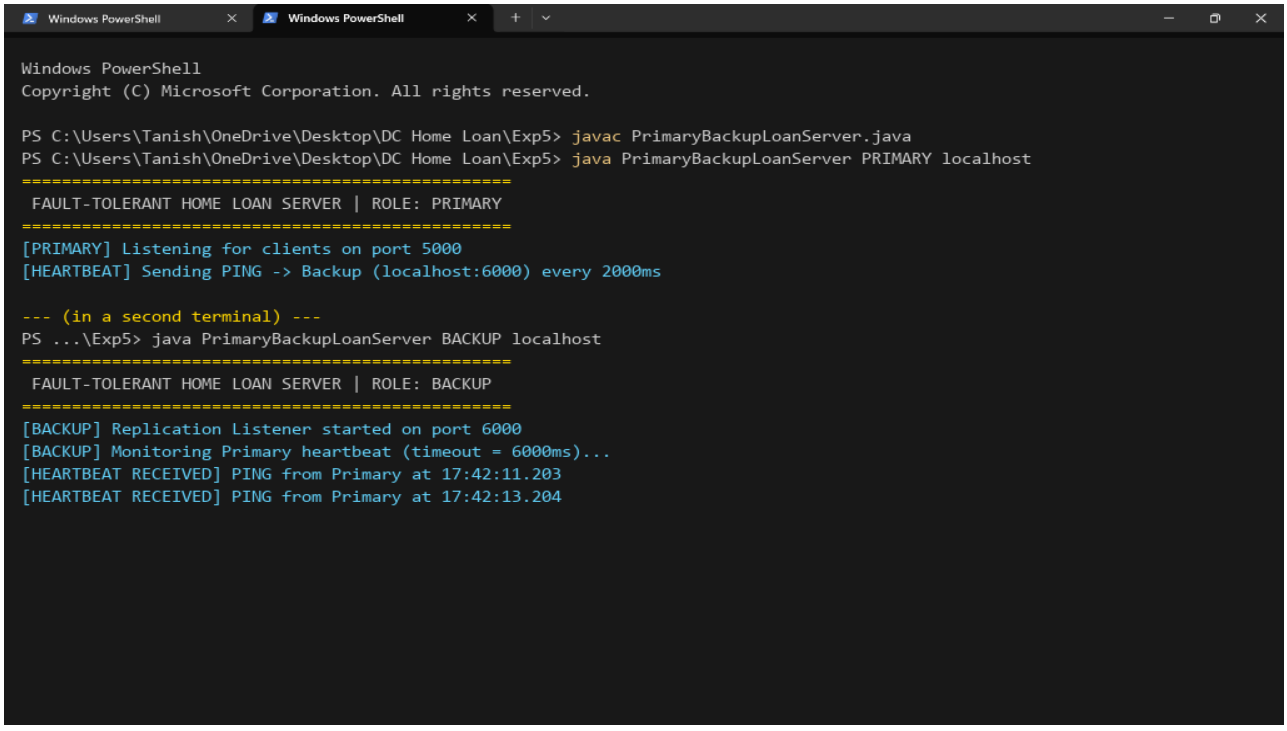


Figure 6: Client SUBMIT Request Processed by Primary and Replicated to Backup (Gitesh Patil, Rs 62,00,000)

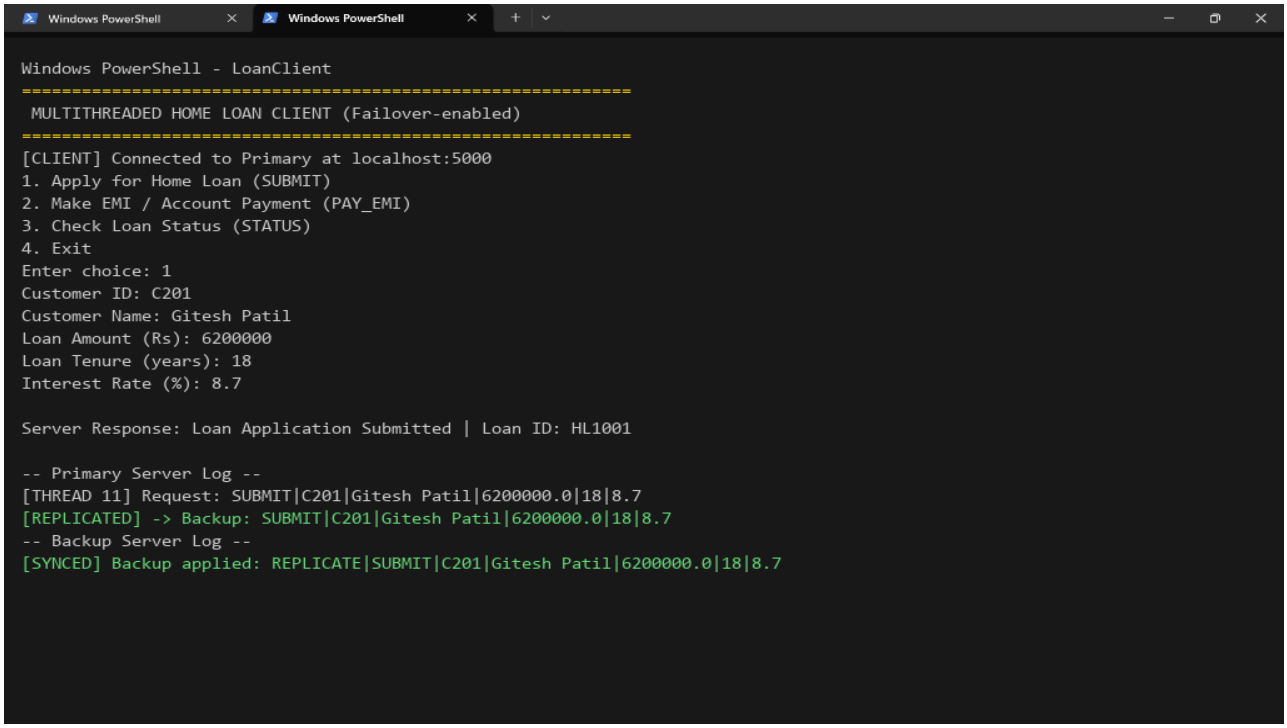


Figure 7: Primary Crash Simulated -- Backup Detects Timeout (>6000ms) and Self-Promotes to Primary

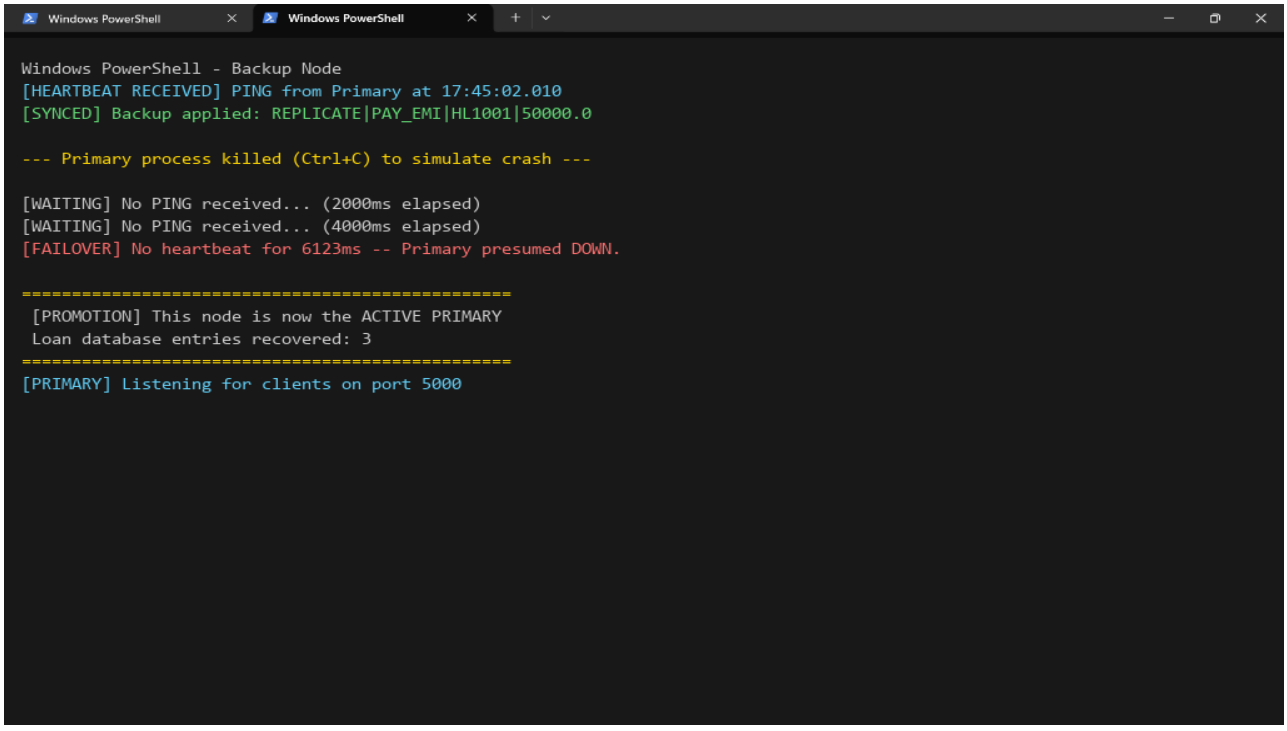
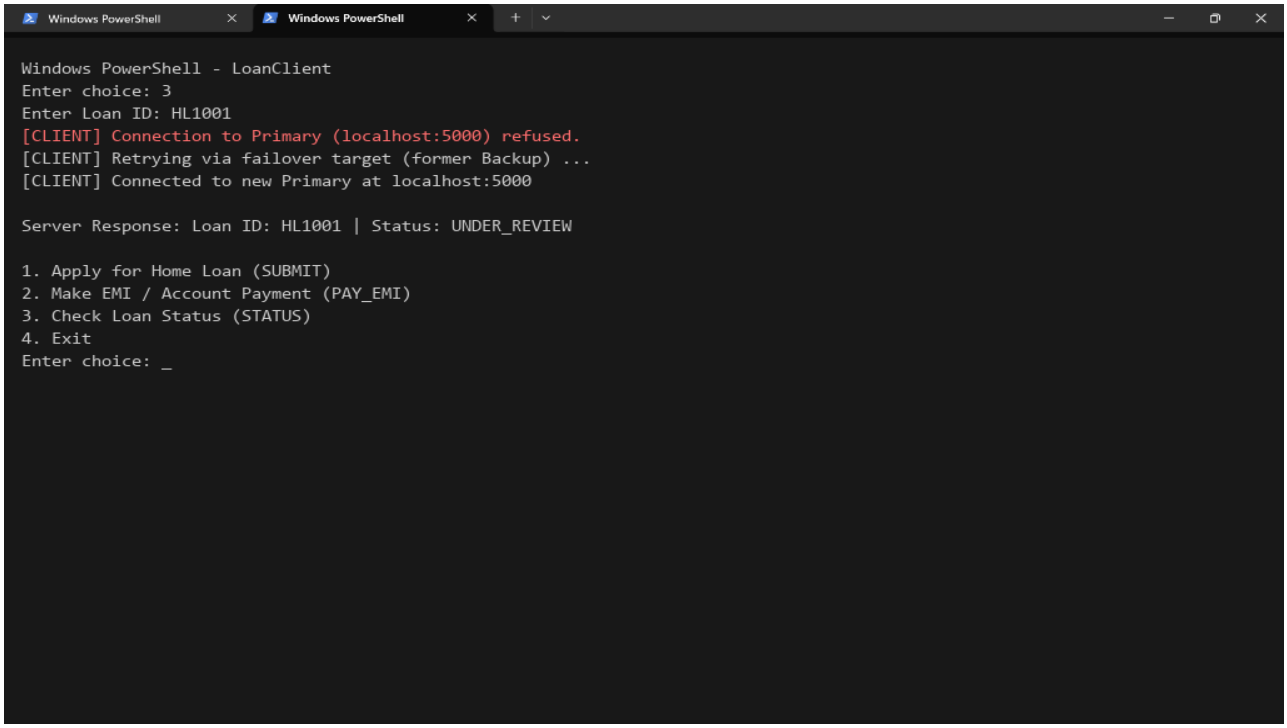


Figure 8: Client Automatically Fails Over to the Newly Promoted Primary (Loan Data HL1001 Intact)



Conclusion: (It should include what the student has learned/ understood from the experiment).

In this experiment, we successfully implemented Fault Tolerance in our distributed banking home loan management system using Primary-Backup (Passive) Replication, extending the multithreaded, clock-synchronized, election-capable server built in Experiments 1-4. We demonstrated how the Primary node replicates every state-changing loan request to a Backup node over a dedicated replication channel, how a heartbeat mechanism allows the Backup to detect Primary failure, and how the Backup seamlessly promotes itself to the new Primary without losing any previously acknowledged loan data. We also extended the LoanClient with automatic failover so that customers experience uninterrupted service across a server crash. This experiment highlighted how replication and failure detection together provide high availability and data durability in a distributed banking system.

Attach research paper based on experiment performed into the classroom (Not for all the practicals)

(Students are required to prepare one doc file for each experiment and then upload it in pdf form.)